AI Assignment

Masabattula Teja Nikhil
Department of Computer Science and Engineering
Indian Institute of Technology Jodhpur
p26cs0004@iitj.ac.in

1 The Problem and the Plan

The task is to determine whether a statement follows from a small knowledge base written in English. The system must use a classical reasoning algorithm rather than a language model to guess the answer. This involves converting the text into a machine-readable form and then applying a formal inference procedure to reach a decision.

The three public datasets used for evaluation are quite different from one another, and this shaped the whole design. **RuleTaker** uses short, template-like sentences ("If something is red then it is round"). **CounterBench** describes chains of causes with invented words and asks counterfactual questions ("Would Lumbo occur if not Ziklo instead of Ziklo?"). **FOLIO** is written in rich, real English with quantifiers, disjunction, and nested conditions, but every example also comes with a first-order logic (FOL) translation. Rather than forcing one representation on all three, the agent uses a single entry point, `predict(context, statement)`, which routes each input to the representation and algorithm that suits it.

Input looks like	Internal representation	Reasoning algorithm
Facts and if–then rules (RuleTaker)	Literals and Horn-style rules with variables	Forward chaining with unification, stratified negation, closed-world assumption
Causal story + "Would Y occur…?" (CounterBench)	Boolean structural equations	Abduction by exhaustive search, then intervention and propagation
Premises with FOL (FOLIO)	Clauses in conjunctive normal form	Resolution refutation, set of support, uniform-cost clause selection

Table 1: One router, three representation/algorithm pairs.

2 The Data

We used the full RuleTaker test split from `tasksource/ruletaker` (151,911 queries over eight configurations, from depth-0 up to depth-5 plus the paraphrased "NatLang" sets), all 1,200 CounterBench questions (basic, conditional, joint, nested and backdoor) and all 1,204 FOLIO v2 examples (train and validation). Since nothing is trained, every public example is used only for evaluation. The assignment calls for a True/False answer, so RuleTaker's *entailment* maps to True, CounterBench's *yes* maps to True, and in FOLIO only the label *True* counts as entailed (*False* and *Uncertain* both mean "not entailed"). RuleTaker and CounterBench are balanced at roughly 50% True; FOLIO is not (about 38% True), which is relevant when

interpreting its accuracy.

3 Representation

3.1 Facts and Rules

A **literal** is a triple `(negated, predicate, arguments)`. "The bear does not chase the dog" becomes `(True, chase, (bear, dog))`, and "Bob is young" becomes `(False, young, (bob,))`. A **rule** is a list of body literals plus one head literal. Words such as *something, someone, it, they, people, things* become the variable X, so "If someone is smart and kind, then they are successful" becomes $\mathrm{smart}(X) \wedge \mathrm{kind}(X) \rightarrow \mathrm{successful}(X)$.

The parser is rule-based. It first recognises the sentence shape: an *if... then...* rule, a group rule such as "All red, big things are kind" or "Kind, young people are not smart", or a plain fact. It then splits conjunctions and fills in what English leaves implicit: in "is big and not red" the second clause borrows both the subject and the verb from the first, giving $\mathrm{big}(X)$ and $\neg\mathrm{red}(X)$. Verbs are reduced to a single form (*chases, chase* $\rightarrow$ `chase`) so that "it chases" and "they chase" refer to the same predicate. On RuleTaker this parser understood 100% of the sentences and 100% of the queries (Section 5 reports the coverage experiment).

3.2 First-Order Clauses

FOLIO's English is well beyond what a template parser can handle: our parser read only 81.5% of its sentences, and even those rarely lined up with the wording of the conclusion. Using that parse alone, the agent scores 62.04%, no better than always answering False (61.79%). Since FOLIO ships an FOL translation for every premise and conclusion, we use those as the representation instead. A recursive-descent parser reads the symbols $\forall\ \exists\ \neg\ \wedge\ \vee\ \rightarrow\ \leftrightarrow\ \oplus$, and every formula is converted to clauses: implications and equivalences are rewritten, negations are pushed inward, existential variables are replaced by Skolem constants or functions, and $\vee$ is distributed over $\wedge$.

3.3 Structural Causal Equations

For CounterBench, "A causes B" becomes $B = A$, "A causes not B" becomes $B = \neg A$, "A and B together cause C" becomes $C = A \wedge B$, "A or B causes C" becomes $C = A \vee B$, and "A causes B, C and D" assigns the same equation to three children. "We observed X" is stored as evidence. The question supplies the **interventions** ("if not X", "if not X and not Z", "assume not X... further suppose not Z") and, for "... instead of X", a **factual** statement that X really happened.

4 Reasoning Algorithms

4.1 Forward Chaining with Unification and Stratified Negation

Forward chaining starts from the facts and keeps firing rules whose bodies are satisfied, adding the heads as new facts, until no new fact appears (a fixpoint). Matching a rule body against the facts uses **unification**: the variable X is bound to a constant that makes every positive body literal true. Each new fact records the rule and the binding that produced it, which is what allows the agent to print a proof afterward.

Two details made the biggest difference on RuleTaker. The first is the closed-world assumption (CWA), where a statement such as "Erin is not big" is considered true if "Erin is big" cannot be derived. The second is rule ordering. For example, the rule "If someone is quiet and not green then they are red" should only fire after all facts about being green have been determined. We handle this by grouping predicates into strata and computing predicates involved in negation first. Some RuleTaker theories contain negation cycles that cannot be stratified, so we apply all rules together until no new facts can be derived. Finally, we answer each query by checking whether it appears in the set of derived facts.

4.2 Resolution Refutation

To prove a FOLIO conclusion Q, we add the clauses of $\neg Q$ to the premises and search for a contradiction, the empty clause. Two clauses are resolved when a literal in one unifies with the negation of a literal in the other, producing a new clause from the remaining literals. To keep the search focused we use the **set-of-support** strategy (every resolution step must involve the negated goal or something derived from it), and we always expand the **smallest clause first**, which is uniform-cost search with clause size as the cost function. A step budget caps the search; if no contradiction is found in time, the answer defaults to False.

4.3 Counterfactual Reasoning as Search

A counterfactual question is answered in three classical steps. Abduction enumerates all True/False assignments to the root variables and keeps only those consistent with the observations and factual statement. Action forces the intervened variables to their new values in each surviving world. Prediction propagates the equations and reads the target variable's value. If the evidence is inconsistent, the system relaxes it step by step, removing the factual statement first and then the observations.

5 Experiments and Results

All numbers below are taken directly from the executed notebook. Time is the wall-clock time of parsing plus reasoning (data loading excluded).

Dataset	**Queries**	**Accuracy (%)**	**Time (s)**	**ms/query**
RuleTaker (test)	151,911	99.97	10.03	0.066
CounterBench	1,200	94.58	0.12	0.096
FOLIO v2 (train + validation)	1,204	80.90	30.58	25.398
All queries	**154,315**	**99.78**	**40.73**	**0.264**

Table 2: Final accuracy and inference time on the public data (notebook, Section 7).

The overall accuracy of 99.78% is largely driven by RuleTaker, which accounts for approximately 98.4% of all queries. Giving each dataset equal weight yields an average accuracy of approximately 91.82%, providing a fairer comparison across datasets. Inference time also varies significantly across reasoning methods. RuleTaker takes 0.066 ms per query, while FOLIO proof search takes 25.398 ms, making it roughly 385 times slower.

RuleTaker group	**Acc. (%)**	**CounterBench type**	**Acc. (%)**	**FOLIO split**	**Acc. (%)**
depth-0	99.99	basic	100.00	train	80.42
depth-1	100.00	conditional	90.00	validation	83.25
depth-2	99.99	joint	96.00		
depth-3	99.95	nested	96.00		
depth-3ext	99.96	backdoor	90.00		
depth-5	99.92				
NatLang	100.00				
depth-3ext-NatLang	99.97				

Table 3: Accuracy by group. Deeper RuleTaker proofs cost almost nothing in accuracy.

5.1 Does the Closed-World Assumption Matter?

We forced the forward chainer into each mode on the whole RuleTaker test set. The result is clear-cut: under the open-world assumption a negated statement is only true if it is explicitly derived, and accuracy on negated queries collapses to 54.84%. The closed-world assumption matches how the dataset was constructed.

Mode	Accuracy (%)	Positive queries (%)	Negated queries (%)	Time (s)
Closed world (CWA)	99.97	99.98	99.96	9.67
Open world (OWA)	74.57	94.51	54.84	9.23

Table 4: CWA versus OWA on RuleTaker (notebook, Section 8.2).

5.2 FOLIO: Representation versus Search Budget

This experiment isolates two effects. First, the representation matters far more than the algorithm: reading FOLIO's English with our rule parser gives baseline-level accuracy, while the FOL route adds about 19 points. Second, the resolution budget is a clean accuracy/time trade-off. Most provable conclusions need only a few dozen steps; going from 1,000 to 3,000 steps gains roughly one additional query but costs five times the time. We therefore use 1,000 steps as the default.

Setting	Accuracy (%)	Time (s)
Always predict False (baseline)	61.79	0.00
NL text only (rule parser + forward chaining)	62.04	0.09
FOL + resolution, max 50 steps	79.73	1.52
FOL + resolution, max 200 steps	80.73	3.63
FOL + resolution, max 1000 steps	80.90	31.78
FOL + resolution, max 3000 steps	80.90	148.29

Table 5: Representation and search budget on FOLIO (notebook, Section 8.3).

5.3 Where the Remaining Errors Come From

The notebook lists 46 RuleTaker errors (28 missed, 18 wrongly accepted), 65 CounterBench errors (45 missed, 20 wrongly accepted), and 230 FOLIO errors (204 missed, 26 wrongly accepted). These fall into three groups.

RuleTaker: the remaining cases are theories where negation sits inside a cycle; the answer then depends on the order in which rules fire, and our order sometimes differs from the dataset generator's.

CounterBench: many errors come from stories that contain a *causal loop*, for example "Blaf and Trune and Vork together cause Sline" together with "Trune or Sline cause not Vork". A loop has no single consistent solution, so propagation cannot settle the target.

FOLIO: 204 of the 230 errors are missed proofs (label True, prediction False). Inspecting them points mostly at the annotations rather than the prover: the same entity sometimes appears under different names in the premises and the conclusion (e.g. `yr1946` vs. `year1946`), or a sentence meaning inclusive "or" is annotated with exclusive-or ($\oplus$). One case is shown in the notebook: "Rina either doesn't want to be addicted…or is unaware…" is labelled True, but its FOL uses $\oplus$. The same conclusion written with $\vee$ is proved in 10 steps (Trace B below).

6 Reasoning Traces

Because every derived fact and every resolvent remembers where it came from, the agent can explain each True answer. The three traces below are copied directly from the notebook output.

A. Forward Chaining (RuleTaker, depth-5)

Query: "The rabbit visits the cat." The tree reads from the conclusion down to the given facts; each line names the rule and the unifier used.

```
visit(rabbit, cat) [rule: kind(X) and like(X, rabbit) -> visit(rabbit, cat) with {X:
    cat}]
  kind(cat) [fact]
  like(cat, rabbit) [rule: chase(X, squirrel) and chase(X, cat) -> like(cat, rabbit)
      with {X: cat}]
    chase(cat, squirrel) [rule: rough(X) and kind(X) -> chase(X, squirrel) with {X:
        cat}]
      rough(cat) [fact]
      kind(cat) [fact]
    chase(cat, cat) [rule: rough(X) and like(X, mouse) -> chase(X, cat) with {X: cat}]
      rough(cat) [fact]
      like(cat, mouse) [rule: cold(X) and visit(X, cat) -> like(cat, mouse) with {X:
          mouse}]
        cold(mouse) [rule: green(X) and visit(X, mouse) -> cold(mouse) with {X: cat}]
          green(cat) [fact]
          visit(cat, mouse) [fact]
        visit(mouse, cat) [fact]
```

A second, short trace shows the closed-world assumption at work. From "If someone is not red then they are not quiet" and no way to derive that Bob is red, the agent concludes "Bob is not quiet":

```
not quiet(bob) [rule: not red(X) -> not quiet(X) with {X: bob}]
  not red(bob) [closed world: red(bob) cannot be derived]
```

B. Resolution Refutation (FOLIO)

Conclusion: $\neg\text{WantToBeAddictedTo}(\text{rina}, \text{caffeine}) \vee \neg\text{AwareThatDrug}(\text{rina}, \text{caffeine})$. Its negation gives two unit clauses, $\text{WantToBeAddictedTo}(\text{rina}, \text{caffeine})$ and $\text{AwareThatDrug}(\text{rina}, \text{caffeine})$, and the prover reaches the empty clause in 10 resolution steps:

```
WantToBeAddictedTo(rina, caffeine) + DrinkRegularly(x, coffee) or not
    WantToBeAddictedTo(x, caffeine)
   => DrinkRegularly(rina, coffee)
DrinkRegularly(rina, coffee) + IsDependentOn(x, caffeine) or not DrinkRegularly(x,
    coffee)
   => IsDependentOn(rina, caffeine)
AwareThatDrug(rina, caffeine) + not AwareThatDrug(rina, caffeine) or not Student(rina
    )
   => not Student(rina)
not Student(rina) + Student(rina) or not IsDependentOn(rina, caffeine)
   => not IsDependentOn(rina, caffeine)
IsDependentOn(rina, caffeine) + not IsDependentOn(rina, caffeine)
   => [] (empty clause)
```

C. Counterfactual Search (CounterBench)

"Ziklo causes not Blaf, Blaf and Trune together cause Vork, Vork causes Lumbo. We observed Trune. Would Lumbo occur if not Ziklo instead of Ziklo?" Abduction keeps the single world where Ziklo happened and

Trune is true; the intervention then flips Ziklo and the effect propagates down the chain:

```
Blaf = not Ziklo
Vork = Blaf and Trune
Lumbo = Vork

exogenous : {Trune: True, Ziklo: True}
factual : {Trune: True, Ziklo: True, Blaf: False, Vork: False, Lumbo: False}
after do(): {Trune: True, Ziklo: False, Blaf: True, Vork: True, Lumbo: True} -> yes (
    gold: yes)
```

7 Conclusion

The agent is fast and nearly exact where the language is controlled, and it is transparent about how it reaches each answer. Its main weakness is the step from English to logic when the English is free-form: on FOLIO it relies on the provided FOL, and if a test example arrives without FOL, the agent falls back to the rule parser, which on FOLIO-style text performs at baseline level (62.04%). Causal stories with loops and RuleTaker theories with negation inside a cycle are the other known gaps.

Overall, the experiments support the central design choice that a good representation enables effective classical reasoning. Once the text is in the right form, forward chaining answers 151,911 RuleTaker queries at 99.97% accuracy in about seven seconds, a small exhaustive search handles CounterBench at 94.58%, and resolution reaches 80.90% on FOLIO. Every answer accompanied by a proof that can be checked by hand.